

Docker Lab Submission

Name: Ayush Patel

Roll No: 202301084

All steps below were performed on a Linux machine (user ayushpatel) with Docker Engine 29.6.2 and Docker Compose v5.3.1. Every screenshot is the real output of the commands shown above it.

Project Files

Dockerfile

```
# Base image: lightweight Nginx web server on Alpine Linux
FROM nginx:alpine

# Metadata about the image author
LABEL maintainer="Ayush Patel (202301084)"

# Copy our web page into Nginx's default web root
COPY app/index.html /usr/share/nginx/html/index.html

# Document that the container listens on port 80
EXPOSE 80

# Start Nginx in the foreground so the container keeps running
CMD ["nginx", "-g", "daemon off;"]
```

docker-compose.yml

```
services:
  web:
    build: .
    image: ayushpatel/docker-lab-web:1.0
    container_name: ayushpatel-compose-web
    ports:
      - "8081:80"
    volumes:
      - web-logs:/var/log/nginx
    restart: unless-stopped

volumes:
  web-logs:
```

app/index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Docker Lab - Ayush Patel</title>
  <style>
    body { font-family: Arial, sans-serif; background: #fff; color: #000;
      display: flex; align-items: center; justify-content: center; height: 100vh; margin: 0; }
    .card { background: #fff; color: #000; padding: 40px 60px; border: 2px solid #000; text-align: center; }
    h1 { color: #000; margin-top: 0; }
  </style>
</head>
<body>
  <div class="card">
    <h1>Hello from Docker!</h1>
    <p><b>Name:</b> Ayush Patel</p>
    <p><b>Roll No:</b> 202301084</p>
    <p>This page is served by an Nginx container built from my own Dockerfile.</p>
  </div>
</body>
</html>
```

Step 1: Check Docker installation

Command	Purpose
docker --version	Shows the installed Docker client version, confirming Docker is installed.
docker compose version	Shows the Docker Compose plugin version (used later for multi-container deployment).
docker info --format 'Server: {{.ServerVersion}} Containers: {{.Containers}} Images: {{.Images}} OS: {{.OperatingSystem}}'	Prints key details about the Docker daemon (engine version, number of containers/images, OS) to confirm the daemon is running.

```
ayushpatel@docker-lab: ~/docker-lab - Step 1: Check Docker installation

ayushpatel@docker-lab:~/docker-lab$ docker --version
Docker version 29.6.2, build dfc4efb
ayushpatel@docker-lab:~/docker-lab$ docker compose version
Docker Compose version v5.3.1
ayushpatel@docker-lab:~/docker-lab$ docker info --format 'Server: {{.ServerVersion}} | Containers: {{.Containers}} | Images: {{.Images}} | OS:
> {{.OperatingSystem}}'
Server: 29.6.2 | Containers: 0 | Images: 0 | OS: Ubuntu 24.04.4 LTS
ayushpatel@docker-lab:~/docker-lab$
```

Step 2: Run the first container (hello-world)

Command	Purpose
<code>docker run hello-world</code>	Downloads the hello-world image (if not present), creates a container from it and runs it. The container prints a message and exits - proving Docker works end to end.

```
ayushpatel@docker-lab: ~/docker-lab  -  Step 2: Run the first container (hello-world)

ayushpatel@docker-lab:~/docker-lab$ docker run hello-world

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
   (amd64)
3. The Docker daemon created a new container from that image which runs the
   executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it
   to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

For more examples and ideas, visit:
https://docs.docker.com/get-started/

Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
4f55086f7dd0: Pulling fs layer
d5e71e642bf5: Download complete
4f55086f7dd0: Download complete
4f55086f7dd0: Pull complete
Digest: sha256:5e23090353324d887c48ad5e5c56d294eab81588df9605b07d1afe895f9cc8f8
Status: Downloaded newer image for hello-world:latest
ayushpatel@docker-lab:~/docker-lab$
```

Step 3: Pull images and list them

Command	Purpose
docker pull nginx:alpine	Downloads the official Nginx web-server image (small Alpine variant) from Docker Hub.
docker pull alpine:latest	Downloads the tiny Alpine Linux base image.
docker images	Lists all images stored locally with their repository, tag, ID and size.

```
ayushpatel@docker-lab: ~/docker-lab  -  Step 3: Pull images and list them

ayushpatel@docker-lab:~/docker-lab$ docker pull nginx:alpine
alpine: Pulling from library/nginx
e72112c14215: Pulling fs layer
d9aae54b5831: Pulling fs layer
9a9a644fdd6a: Pulling fs layer
6c53d0b2a666: Pulling fs layer
e76228b47809: Pulling fs layer
e2de96513ba9: Pulling fs layer
64c8194480fe: Pulling fs layer
745dfb2690dd: Pulling fs layer
9a9a644fdd6a: Download complete
64c8194480fe: Download complete
0deea27e58d9: Download complete
e76228b47809: Download complete
d9aae54b5831: Download complete
745dfb2690dd: Download complete
d54d3939625e: Download complete
6c53d0b2a666: Download complete
e2de96513ba9: Download complete
e2de96513ba9: Pull complete
e72112c14215: Download complete
d9aae54b5831: Pull complete
6c53d0b2a666: Pull complete
9a9a644fdd6a: Pull complete
64c8194480fe: Pull complete
e76228b47809: Pull complete
745dfb2690dd: Pull complete
e72112c14215: Pull complete
Digest: sha256:df221db836e1754089190208cee7eeda94f233197056426eda74a43ab1abeac2
Status: Downloaded newer image for nginx:alpine
docker.io/library/nginx:alpine
ayushpatel@docker-lab:~/docker-lab$ docker pull alpine:latest
latest: Pulling from library/alpine
797dd00a0fc7: Download complete
6d0606d1815c: Download complete
Digest: sha256:294b683cb724975bec92580e1e685676bd4b50bda910ddb8c51d4cabeaec77e6
Status: Downloaded newer image for alpine:latest
docker.io/library/alpine:latest
ayushpatel@docker-lab:~/docker-lab$ docker images
IMAGE          ID                DISK USAGE  CONTENT SIZE  EXTRA
alpine:latest  294b683cb724      13MB        3.94MB
hello-world:latest  5e2309035332    25.9kB      9.49kB      U
nginx:alpine    df221db836e1     94.4MB      27.2MB
```

Step 4: Run a command inside a container

Command	Purpose
docker run --rm alpine cat /etc/os-release	Starts an Alpine container, runs one command inside it (shows the container's OS) and removes the container afterwards (--rm).
docker run --rm alpine sh -c 'echo Hello from Ayush Patel, Roll No 202301084; hostname'	Runs a small shell script inside a container. The hostname printed is the container ID, showing each container is an isolated environment. (Interactive form: docker run -it alpine sh)

```
ayushpatel@docker-lab: ~/docker-lab  -  Step 4: Run a command inside a container

ayushpatel@docker-lab:~/docker-lab$ docker run --rm alpine cat /etc/os-release
NAME="Alpine Linux"
ID=alpine
VERSION_ID=3.24.2
PRETTY_NAME="Alpine Linux v3.24"
HOME_URL="https://alpinelinux.org/"
BUG_REPORT_URL="https://gitlab.alpinelinux.org/alpine/aports/-/issues"
ayushpatel@docker-lab:~/docker-lab$ docker run --rm alpine sh -c 'echo Hello from Ayush Patel, Roll No 202301084; hostname'
Hello from Ayush Patel, Roll No 202301084
c75ddb2148a7
ayushpatel@docker-lab:~/docker-lab$
```

Step 5: Run a container in background with port mapping

Command	Purpose
docker run -d --name ayushpatel-nginx -p 8080:80 nginx:alpine	Starts Nginx in detached mode (-d, background), names it ayushpatel-nginx and maps host port 8080 to container port 80 (-p).
docker ps	Lists running containers with ID, image, status and port mappings.
curl -s http://localhost:8080 grep -i '<title>'	Sends an HTTP request to the container through the mapped port to prove the web server is reachable.

```
ayushpatel@docker-lab: ~/docker-lab - Step 5: Run a container in background with port mapping

ayushpatel@docker-lab:~/docker-lab$ docker run -d --name ayushpatel-nginx -p 8080:80 nginx:alpine
8b4fc9af86ebb746de370df51f0db2ebddf51752807c38a31bb1ba83c2dd355d
ayushpatel@docker-lab:~/docker-lab$ docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS                               NAMES
8b4fc9af86eb   nginx:alpine  "/docker-entrypoint..."  3 seconds ago  Up 2 seconds  0.0.0.0:8080->80/tcp              ayushpatel-nginx
ayushpatel@docker-lab:~/docker-lab$ curl -s http://localhost:8080 | grep -i '<title>'
<title>Welcome to nginx!</title>
ayushpatel@docker-lab:~/docker-lab$
```

Step 6: Logs, exec and inspect

Command	Purpose
docker logs ayushpatel-nginx 2>&1 tail -n 4	Shows the output/logs written by the container (here the last 4 lines, including our curl request).
docker exec ayushpatel-nginx ls -l /usr/share/nginx/html	Runs a command inside the already-running container (docker exec) - here listing Nginx's web folder.
docker inspect --format 'Name={{.Name}} IP={{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}} Status={{.State.Status}}' ayushpatel-nginx	Shows low-level details of the container (name, internal IP address, state) using a Go template filter.

```
ayushpatel@docker-lab: ~/docker-lab - Step 6: Logs, exec and inspect

ayushpatel@docker-lab:~/docker-lab$ docker logs ayushpatel-nginx 2>&1 | tail -n 4
2026/10/03 17:24:29 [notice] 1#1: start worker process 21
2026/10/03 17:24:29 [notice] 1#1: start worker process 22
2026/10/03 17:24:29 [notice] 1#1: start worker process 23
172.17.0.1 - - [03/Oct/2026:17:24:34 +0000] "GET / HTTP/1.1" 200 896 "-" "curl/8.5.0" "-"
ayushpatel@docker-lab:~/docker-lab$ docker exec ayushpatel-nginx ls -l /usr/share/nginx/html
total 8
-rw-r--r-- 1 root root 497 Sep 15 14:18 50x.html
-rw-r--r-- 1 root root 896 Sep 15 14:18 index.html
ayushpatel@docker-lab:~/docker-lab$ docker inspect --format 'Name={{.Name}} IP={{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}} \
> Status={{.State.Status}}' ayushpatel-nginx
Name=/ayushpatel-nginx IP=172.17.0.2 Status=running
ayushpatel@docker-lab:~/docker-lab$
```

Step 7: Container lifecycle: stop, start, remove

Command	Purpose
docker stop ayushpatel-nginx	Gracefully stops the running container.
docker ps -a --filter name=ayushpatel-nginx	Lists all containers including stopped ones (-a); status is now Exited.
docker start ayushpatel-nginx	Starts the stopped container again (same container, same settings).
docker ps --filter name=ayushpatel-nginx	Confirms the container is Up again.
docker rm -f ayushpatel-nginx	Force-removes the container (stops it first because of -f).

```
ayushpatel@docker-lab: ~/docker-lab - Step 7: Container lifecycle: stop, start, remove

ayushpatel@docker-lab:~/docker-lab$ docker stop ayushpatel-nginx
ayushpatel-nginx
ayushpatel@docker-lab:~/docker-lab$ docker ps -a --filter name=ayushpatel-nginx
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS              PORTS          NAMES
8b4fc9af86eb   nginx:alpine  "/docker-entrypoint..."  16 seconds ago  Exited (0) 2 seconds ago           ayushpatel-nginx
ayushpatel@docker-lab:~/docker-lab$ docker start ayushpatel-nginx
ayushpatel-nginx
ayushpatel@docker-lab:~/docker-lab$ docker ps --filter name=ayushpatel-nginx
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS              PORTS          NAMES
8b4fc9af86eb   nginx:alpine  "/docker-entrypoint..."  21 seconds ago  Up 2 seconds       0.0.0.0:8080->80/tcp  ayushpatel-nginx
ayushpatel@docker-lab:~/docker-lab$ docker rm -f ayushpatel-nginx
ayushpatel-nginx
ayushpatel@docker-lab:~/docker-lab$
```

Step 8: Write a Dockerfile

Command	Purpose
ls -R	Shows the project files: the Dockerfile, the web page in app/, and the Compose file.
cat Dockerfile	Displays the Dockerfile. FROM picks the base image, LABEL adds metadata, COPY adds our page, EXPOSE documents the port, CMD sets the start command.

```
ayushpatel@docker-lab: ~/docker-lab  -  Step 8: Write a Dockerfile

ayushpatel@docker-lab:~/docker-lab$ ls -R
.:
Dockerfile
app
docker-compose.yml

./app:
index.html
ayushpatel@docker-lab:~/docker-lab$ cat Dockerfile
# Base image: lightweight Nginx web server on Alpine Linux
FROM nginx:alpine

# Metadata about the image author
LABEL maintainer="Ayush Patel (202301084)"

# Copy our web page into Nginx's default web root
COPY app/index.html /usr/share/nginx/html/index.html

# Document that the container listens on port 80
EXPOSE 80

# Start Nginx in the foreground so the container keeps running
CMD ["nginx", "-g", "daemon off;"]
ayushpatel@docker-lab:~/docker-lab$
```

Step 9: Build a custom image

Command	Purpose
docker build -t ayushpatel/docker-lab-web:1.0 .	Builds a new image from the Dockerfile in the current folder (.) and tags it (-t) as ayushpatel/docker-lab-web:1.0.
docker images ayushpatel/docker-lab-web	Lists the newly built image.
docker history ayushpatel/docker-lab-web:1.0 head -n 6	Shows the layers of the image; the top layers correspond to our Dockerfile instructions.

```
ayushpatel@docker-lab: ~/docker-lab - Step 9: Build a custom image

ayushpatel@docker-lab:~/docker-lab$ docker build -t ayushpatel/docker-lab-web:1.0 .
#0 building with "default" instance using docker driver

#1 [internal] load build definition from Dockerfile
#1 transferring dockerfile: 457B done
#1 DONE 0.0s

#2 [internal] load metadata for docker.io/library/nginx:alpine
#2 DONE 0.0s

#3 [internal] load .dockerignore
#3 transferring context: 69B done
#3 DONE 0.0s

#4 [internal] load build context
#4 transferring context: 769B done
#4 DONE 0.0s

#5 [1/2] FROM docker.io/library/nginx:alpine@sha256:df221db836e1754089190208cee7eeda94f233197056426eda74a43ab1abeac2
#5 resolve docker.io/library/nginx:alpine@sha256:df221db836e1754089190208cee7eeda94f233197056426eda74a43ab1abeac2 0.0s done
#5 DONE 0.1s

#6 [2/2] COPY app/index.html /usr/share/nginx/html/index.html
#6 DONE 0.0s

#7 exporting to image
#7 exporting layers 0.1s done
#7 exporting manifest sha256:3a9f385b5fcl5f0913293e2a5725689d21b6716f618a5066b50e34625a09c90 done
#7 exporting config sha256:47c57b84f329b731f0ecda920dec5fa8f87e120a8416cb6e7f1003241fc6238d done
#7 exporting attestation manifest sha256:490455472a6950402eee779fcee9a0b57079ad355c40487945a9e6b7e1d53d05 0.0s done
#7 exporting manifest list sha256:1e1b310b336a8f6f2c6a78fc5147884fada071255ad811ffcad24b7e697af8d3
#7 exporting manifest list sha256:1e1b310b336a8f6f2c6a78fc5147884fada071255ad811ffcad24b7e697af8d3 done
#7 naming to docker.io/ayushpatel/docker-lab-web:1.0 done
#7 unpacking to docker.io/ayushpatel/docker-lab-web:1.0 0.0s done
#7 DONE 0.2s
ayushpatel@docker-lab:~/docker-lab$ docker images ayushpatel/docker-lab-web
IMAGE ID DISK USAGE CONTENT SIZE EXTRA
ayushpatel/docker-lab-web:1.0 1e1b310b336a 93.6MB 26.3MB
ayushpatel@docker-lab:~/docker-lab$ docker history ayushpatel/docker-lab-web:1.0 | head -n 6
IMAGE CREATED CREATED BY SIZE COMMENT
1e1b310b336a 5 seconds ago CMD ["nginx" "-g" "daemon off;"] 0B buildkit.dockerfile.v0
<missing> 5 seconds ago EXPOSE [80/tcp] 0B buildkit.dockerfile.v0
<missing> 5 seconds ago COPY app/index.html /usr/share/nginx/html/in... 24.6kB buildkit.dockerfile.v0
<missing> 5 seconds ago LABEL maintainer=Ayush Patel (202301084) 0B buildkit.dockerfile.v0
<missing> 10 days ago RUN /bin/sh -c set -x && apkArch="$(cat ... 52.4MB buildkit.dockerfile.v0
ayushpatel@docker-lab:~/docker-lab$
```

Step 10: Run the custom image (deployment)

Command	Purpose
docker run -d --name ayushpatel-web -p 8082:80 ayushpatel/docker-lab-web:1.0	Deploys our own image as a container on host port 8082.
docker ps --filter name=ayushpatel-web	Confirms the container is running.
curl -s http://localhost:8082 grep -E 'Name Roll'	Fetches the page from the container - it shows my name and roll number, proving the custom image works.

```
ayushpatel@docker-lab: ~/docker-lab - Step 10: Run the custom image (deployment)

ayushpatel@docker-lab:~/docker-lab$ docker run -d --name ayushpatel-web -p 8082:80 ayushpatel/docker-lab-web:1.0
e63bdbbd9dea6a39deeb2ee51ef5a2d563cf9073ad6904ad17f38c479a48c48b
ayushpatel@docker-lab:~/docker-lab$ docker ps --filter name=ayushpatel-web
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS                               NAMES
e63bdbbd9dea   ayushpatel/docker-lab-web:1.0       "/docker-entrypoint..." 2 seconds ago  Up 2 seconds  0.0.0.0:8082->80/tcp              ayushpatel-web
ayushpatel@docker-lab:~/docker-lab$ curl -s http://localhost:8082 | grep -E 'Name|Roll'
<p><b>Name:</b> Ayush Patel</p>
<p><b>Roll No:</b> 202301084</p>
ayushpatel@docker-lab:~/docker-lab$
```

Browser view of the custom image running with docker run (port 8082).

http://localhost:8082/

Hello from Docker!

Name: Ayush Patel

Roll No: 202301084

This page is served by an Nginx container built from my own Dockerfile.

Step 11: Named volumes (persistent data)

Command	Purpose
<code>docker volume create ayush-data</code>	Creates a named volume - storage managed by Docker that lives outside any container.
<code>docker volume ls</code>	Lists all volumes.
<code>docker run --rm -v ayush-data:/data alpine sh -c "echo 'Ayush Patel - 202301084' > /data/student.txt"</code>	Container #1 mounts the volume at /data and writes a file, then is deleted (--rm).
<code>docker run --rm -v ayush-data:/data alpine cat /data/student.txt</code>	Container #2 (a brand new one) mounts the same volume and reads the file - data persisted after container #1 was removed.
<code>docker volume inspect ayush-data</code>	Shows volume details such as where Docker stores it on the host (Mountpoint).

ayushpatel@docker-lab: ~/docker-lab - Step 11: Named volumes (persistent data)

```
ayushpatel@docker-lab:~/docker-lab$ docker volume create ayush-data
ayush-data
ayushpatel@docker-lab:~/docker-lab$ docker volume ls
DRIVER      VOLUME NAME
local       ayush-data
ayushpatel@docker-lab:~/docker-lab$ docker run --rm -v ayush-data:/data alpine sh -c "echo 'Ayush Patel - 202301084' > /data/student.txt"
ayushpatel@docker-lab:~/docker-lab$ docker run --rm -v ayush-data:/data alpine cat /data/student.txt
Ayush Patel - 202301084
ayushpatel@docker-lab:~/docker-lab$ docker volume inspect ayush-data
[
  {
    "CreatedAt": "2026-10-03T17:25:13Z",
    "Driver": "local",
    "Labels": null,
    "Mountpoint": "/var/lib/docker/volumes/ayush-data/_data",
    "Name": "ayush-data",
    "Options": null,
    "Scope": "local"
  }
]
```

Step 12: Bind mount (host folder into container)

Command	Purpose
<code>mkdir -p site && echo '<h1>Bind mount page by Ayush Patel (202301084)</h1>' > site/index.html</code>	Creates a folder on the host with a simple HTML page.
<code>docker run -d --name ayushpatel-bind -p 8083:80 -v "\$(pwd)/site":/usr/share/nginx/html:ro nginx:alpine</code>	Runs Nginx and mounts the host folder into the container's web root (read-only, :ro).
<code>curl -s http://localhost:8083</code>	The container serves the host file.
<code>echo '<h1>Updated on the host - no rebuild needed!</h1>' > site/index.html && curl -s http://localhost:8083</code>	Edit the file on the host; the container immediately serves the new content because it is the same folder.

```
ayushpatel@docker-lab: ~/docker-lab - Step 12: Bind mount (host folder into container)

ayushpatel@docker-lab:~/docker-lab$ mkdir -p site && echo '<h1>Bind mount page by Ayush Patel (202301084)</h1>' > site/index.html
ayushpatel@docker-lab:~/docker-lab$ docker run -d --name ayushpatel-bind -p 8083:80 -v "$(pwd)/site":/usr/share/nginx/html:ro nginx:alpine
cab4eaf7ba09ad097b7508c4316aa41e2396c1f277805ec593f6a76fdd98ba93
ayushpatel@docker-lab:~/docker-lab$ curl -s http://localhost:8083
<h1>Bind mount page by Ayush Patel (202301084)</h1>
ayushpatel@docker-lab:~/docker-lab$ echo '<h1>Updated on the host - no rebuild needed!</h1>' > site/index.html && curl -s http://localhost:8083
<h1>Updated on the host - no rebuild needed!</h1>
ayushpatel@docker-lab:~/docker-lab$
```

Step 13: Docker networks

Command	Purpose
docker network create ayush-net	Creates a user-defined bridge network so containers can talk to each other by name.
docker run -d --name ayush-server --network ayush-net nginx:alpine	Starts a web server container attached to the network.
docker run --rm --network ayush-net alpine wget -q0- http://ayush-server grep -i '<title>'	A second container on the same network reaches the server using its container name (built-in DNS).
docker network ls	Lists networks (bridge, host, none and our ayush-net).

```
ayushpatel@docker-lab: ~/docker-lab  -  Step 13: Docker networks

ayushpatel@docker-lab:~/docker-lab$ docker network create ayush-net
9345d12149ee8518ab31977e3f5b06ee230ff26d8ae727ee7141be2e4483c638
ayushpatel@docker-lab:~/docker-lab$ docker run -d --name ayush-server --network ayush-net nginx:alpine
56dcc4c772321a9753fcf0f6e9f70e0e7c6a4e3faf3fb328da651fced84d590d
ayushpatel@docker-lab:~/docker-lab$ docker run --rm --network ayush-net alpine wget -q0- http://ayush-server | grep -i '<title>'
<title>Welcome to nginx!</title>
ayushpatel@docker-lab:~/docker-lab$ docker network ls
NETWORK ID      NAME      DRIVER      SCOPE
9345d12149ee    ayush-net  bridge      local
64d61954f294    bridge    bridge      local
f4eee35c36b5    host      host        local
62647d20a688    none      null        local
ayushpatel@docker-lab:~/docker-lab$
```

Step 14: Tag and save an image

Command	Purpose
docker tag ayushpatel/docker-lab-web:1.0 ayushpatel/docker-lab-web:latest	Adds another tag (name) to the same image - this is how images are prepared for pushing to a registry (docker login + docker push).
docker images ayushpatel/docker-lab-web	Both tags point to the same IMAGE ID.
docker save -o docker-lab-web.tar ayushpatel/docker-lab-web:1.0 && ls -lh docker-lab-web.tar	Exports the image to a .tar file so it can be copied to another machine and loaded with docker load.

```
ayushpatel@docker-lab: ~/docker-lab - Step 14: Tag and save an image

ayushpatel@docker-lab:~/docker-lab$ docker tag ayushpatel/docker-lab-web:1.0 ayushpatel/docker-lab-web:latest
ayushpatel@docker-lab:~/docker-lab$ docker images ayushpatel/docker-lab-web
IMAGE                                ID                                DISK USAGE  CONTENT SIZE  EXTRA
ayushpatel/docker-lab-web:1.0       1e1b310b336a                      93.6MB      26.3MB       U
ayushpatel/docker-lab-web:latest    1e1b310b336a                      93.6MB      26.3MB       U
ayushpatel@docker-lab:~/docker-lab$ docker save -o docker-lab-web.tar ayushpatel/docker-lab-web:1.0 && ls -lh docker-lab-web.tar
-rw----- 1 ayushpatel ayushpatel 26M Oct  3 17:25 docker-lab-web.tar
ayushpatel@docker-lab:~/docker-lab$
```

Step 15: Deploy with Docker Compose

Command	Purpose
cat docker-compose.yml	Shows the Compose file: one service built from our Dockerfile, port 8081, and a named volume for Nginx logs.
docker compose up -d --build	Builds the image and starts all services in the background with a single command.
docker compose ps	Lists the containers managed by this Compose project.
curl -s http://localhost:8081 grep -E 'Name Roll'	Verifies the Compose deployment is serving the page.

```

ayushpatel@docker-lab:~/docker-lab$ cat docker-compose.yml
services:
  web:
    build: .
    image: ayushpatel/docker-lab-web:1.0
    container_name: ayushpatel-compose-web
    ports:
      - "8081:80"
    volumes:
      - web-logs:/var/log/nginx
    restart: unless-stopped

volumes:
  web-logs:
ayushpatel@docker-lab:~/docker-lab$ docker compose up -d --build
#1 [internal] load local bake definitions
#1 reading from stdin 515B done
#1 DONE 0.0s

#2 [internal] load build definition from Dockerfile
#2 transferring dockerfile: 457B done
#2 DONE 0.0s

#3 [internal] load metadata for docker.io/library/nginx:alpine
#3 DONE 0.0s

#4 [internal] load .dockerignore
#4 transferring context: 32B
#4 transferring context: 69B done
#4 DONE 0.0s

#5 [internal] load build context
#5 transferring context: 59B done
#5 DONE 0.0s

#6 [1/2] FROM docker.io/library/nginx:alpine@sha256:df221db836e1754089190208cee7eeda94f233197056426eda74a43ab1abeac2
#6 resolve docker.io/library/nginx:alpine@sha256:df221db836e1754089190208cee7eeda94f233197056426eda74a43ab1abeac2 0.0s done
... (5 lines trimmed) ...
#8 exporting to image
#8 exporting layers done
#8 exporting manifest sha256:5c1fa71f687a1ed2fa0382d9b2621f0ca64835a5a7d4e1bf7d89bd7fd45d5fe7 done
#8 exporting config sha256:bdc615df85ede5906632f24c4f4cf577aa5d64913fab8bd38dc3a0c73bc8788 done
#8 exporting attestation manifest sha256:61fcb1c6043745b34e4a868c99eacd98d6c89f36alf890562abfa6daa60fed61 done
#8 exporting manifest list sha256:db23807fe4a5b09aa88807222af508ce6fbc109d207df7f645c7f01f9a512f6c done
#8 naming to docker.io/ayushpatel/docker-lab-web:1.0 done
#8 unpacking to docker.io/ayushpatel/docker-lab-web:1.0 0.0s done
#8 DONE 0.1s

#9 resolving provenance for metadata file
#9 DONE 0.0s
Image ayushpatel/docker-lab-web:1.0 Building
Image ayushpatel/docker-lab-web:1.0 Built
Network docker-lab_default Creating
Network docker-lab_default Created
Volume docker-lab_web-logs Creating
Volume docker-lab_web-logs Created
Container ayushpatel-compose-web Creating
Container ayushpatel-compose-web Created
Container ayushpatel-compose-web Starting
Container ayushpatel-compose-web Started
ayushpatel@docker-lab:~/docker-lab$ docker compose ps

```

NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS	PORTS
ayushpatel-compose-web	ayushpatel/docker-lab-web:1.0	"/docker-entrypoint..."	web	3 seconds ago	Up 2 seconds	0.0.0.0:8081->80/tcp

```

ayushpatel@docker-lab:~/docker-lab$ curl -s http://localhost:8081 | grep -E 'Name|Roll'
<p><b>Name:</b> Ayush Patel</p>
<p><b>Roll No:</b> 202301084</p>
ayushpatel@docker-lab:~/docker-lab$

```

Browser view of the same app deployed with Docker Compose (port 8081).

http://localhost:8081/

Hello from Docker!

Name: Ayush Patel

Roll No: 202301084

This page is served by an Nginx container built from my own Dockerfile.

Step 16: Monitoring: stats and compose logs

Command	Purpose
<code>docker stats --no-stream --format 'table {{.Name}}\t{{.CPUPerc}}\t{{.MemUsage}}'</code>	Shows a one-time snapshot of CPU and memory usage of running containers.
<code>docker compose logs --tail 3</code>	Shows the last log lines of the Compose services.
<code>docker system df</code>	Shows how much disk space images, containers and volumes use.

```
ayushpatel@docker-lab: ~/docker-lab  —  Step 16: Monitoring: stats and compose logs

ayushpatel@docker-lab:~/docker-lab$ docker stats --no-stream --format 'table {{.Name}}\t{{.CPUPerc}}\t{{.MemUsage}}'
NAME                CPU %    MEM USAGE / LIMIT
ayushpatel-compose-web  0.00%   5.129MiB / 15.72GiB
ayush-server         0.00%   4.605MiB / 15.72GiB
ayushpatel-bind       0.00%   4.582MiB / 15.72GiB
ayushpatel-web        0.00%   4.598MiB / 15.72GiB
ayushpatel@docker-lab:~/docker-lab$ docker compose logs --tail 3
ayushpatel-compose-web | 2026/10/03 17:25:53 [notice] 1#1: start worker process 22
ayushpatel-compose-web | 2026/10/03 17:25:53 [notice] 1#1: start worker process 23
ayushpatel-compose-web | 172.19.0.1 - - [03/Oct/2026:17:25:58 +0000] "GET / HTTP/1.1" 200 699 "-" "curl/8.5.0" "-"
ayushpatel@docker-lab:~/docker-lab$ docker system df
TYPE                TOTAL    ACTIVE    SIZE      RECLAIMABLE
Images              5        4        94.64MB   88.02kB (0%)
Containers          5        4        217.1kB   4.096kB (1%)
Local Volumes       2        1         46B      24B (52%)
Build Cache        12        0       26.38MB   28.67kB
ayushpatel@docker-lab:~/docker-lab$
```

Step 17: Clean up

Command	Purpose
docker compose down -v	Stops and removes the Compose containers, network and volume (-v).
docker rm -f ayushpatel-web ayushpatel-bind ayush-server	Removes the remaining containers.
docker network rm ayush-net && docker volume rm ayush-data	Removes the custom network and volume.
docker container prune -f	Deletes all stopped containers (e.g. the hello-world one).
docker ps -a	Confirms no containers are left.

```
ayushpatel@docker-lab: ~/docker-lab - Step 17: Clean up

ayushpatel@docker-lab:~/docker-lab$ docker compose down -v
Container ayushpatel-compose-web Stopping
Container ayushpatel-compose-web Stopped
Container ayushpatel-compose-web Removing
Container ayushpatel-compose-web Removed
Volume docker-lab_web-logs Removing
Network docker-lab_default Removing
Volume docker-lab_web-logs Removed
Network docker-lab_default Removed
ayushpatel@docker-lab:~/docker-lab$ docker rm -f ayushpatel-web ayushpatel-bind ayush-server
ayushpatel-web
ayushpatel-bind
ayush-server
ayushpatel@docker-lab:~/docker-lab$ docker network rm ayush-net && docker volume rm ayush-data
ayush-net
ayush-data
ayushpatel@docker-lab:~/docker-lab$ docker container prune -f
Deleted Containers:
f02b1274b31054f3d59fc0af0509418d3decd9c35326c30ace17a239db0b1215

Total reclaimed space: 4.096kB
ayushpatel@docker-lab:~/docker-lab$ docker ps -a
CONTAINER ID   IMAGE      COMMAND                  CREATED   STATUS    PORTS   NAMES
ayushpatel@docker-lab:~/docker-lab$
```

Conclusion

In this lab I installed and verified Docker, ran containers from public images, managed the container lifecycle, wrote a Dockerfile and built my own image, used named volumes and bind mounts for data, connected containers with a custom network, tagged/saved the image, and deployed the application with Docker Compose. The browser screenshots confirm the application ran successfully.